



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Reporte proyecto 2: Voyager (Administrador de audioteca digital).

PRESENTA

Bruno Bernardo Soto Lugo
426008646

PROFESOR

Canek Peláez Valdés

AYUDANTES

Miriam Torres Bucio
Cynthia Lizbeth Sánchez Urbano

1. Introducción

El presente documento detalla el proceso de desarrollo de **Voyager**, un sistema diseñado para la gestión y organización de audiotecas locales. El programa permite a los usuarios explorar, minar y visualizar colecciones de archivos de audio, extrayendo automáticamente metadatos como artistas, álbumes, géneros y carátulas de álbumes.

El sistema ha sido desarrollado utilizando el lenguaje **Kotlin** y el framework **Compose Multiplatform**. El desarrollo siguió el modelo de cascada, enfocándose en la definición clara de requisitos, el diseño de arquitectura robusta y la implementación técnica de los componentes de minería y persistencia.

2. Análisis

En esta fase se definieron las capacidades que el sistema debe ofrecer para garantizar una experiencia de usuario eficiente.

2.1. Requisitos

A partir de las especificaciones del programa, se identificaron los siguientes componentes clave:

- **Minería de Archivos:** El sistema debe ser capaz de recorrer directorios de forma recursiva para localizar archivos compatibles. Debe identificar la ruta absoluta y los metadatos asociados.
- **Persistencia de Datos:** Los metadatos extraídos deben almacenarse en una base de datos.
- **Interfaz de Usuario (UI):** El sistema debe presentar una interfaz gráfica que permita navegar por álbumes, visualizar listas de canciones y mostrar detalles específicos de cada pista o artista.
- **Función de búsqueda** El sistema debe presentar una función de búsqueda que permita el uso de filtros.

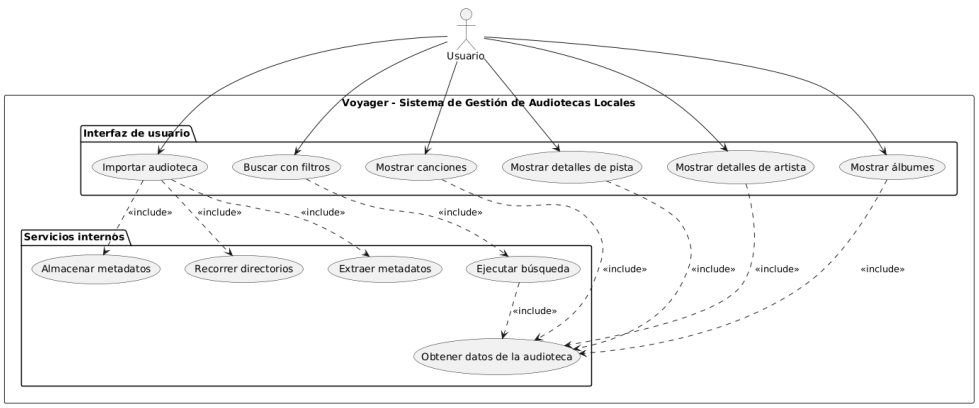


Figura 1: Diagrama de casos de uso de Voyager.

3. Diseño

El diseño de Voyager se centra en la separación de responsabilidades para facilitar la mantenibilidad del código. Para ello, el problema se divide en subproblemas y cada uno se asigna a un componente responsable.

3.1. Descomposición en subproblemas

Subproblema	**Responsabilidad**	**Componente**
Obtención de datos	Recorrer un directorio, localizar los archivos de audio y extraer sus metadatos y carátulas.	`Miner`, `SongMetadata`
Almacenamiento y consulta	Guardar los metadatos y responder las consultas de canciones, álbumes e intérpretes, incluida la búsqueda.	`DatabaseDriver`, entidades (`DbDataClasses`)
Presentación	Mostrar la audioteca, los detalles y los resultados, y capturar las acciones del usuario.	Pantalla principal y columnas
Coordinación	Recibir las acciones del usuario, iniciar la minería, almacenar lo encontrado y mantener el estado que la presentación necesita.	`MainController`

Cuadro 1: Subproblemas de Voyager y el componente responsable de cada uno.

3.2. Patrón arquitectónico: MVC

Los componentes se organizan según el patrón **MVC (Model-View-Controller)**, que separa los datos, su presentación y la lógica que coordina a ambos.

1. **Model:** Agrupa los subproblemas de obtención de datos y de almacenamiento y consulta. El `DatabaseDriver` sigue el patrón DAO: concentra el acceso a los datos para que ningún otro componente conozca cómo se almacenan. El `Miner` no conoce ni el almacenamiento ni la interfaz: avisa de sus resultados mediante las notificaciones `onFound` (por cada canción encontrada) y `onFinish` (al terminar).

2. **View:** Corresponde al subproblema de presentación. Está formada por la pantalla principal, las columnas (navegación, biblioteca de audio, carátulas de álbumes, información de canción y de álbum), los diálogos de edición y búsqueda y el reproductor de audio. Mantiene únicamente el estado visual (canción y álbum seleccionados, panel visible, reproductor activo), pide al usuario el directorio a minar y consulta al `DatabaseDriver` los datos que muestra.

3. **Controller:** Corresponde al subproblema de coordinación. El `MainController` abre la base de datos al crearse, recibe de la vista el directorio elegido (`onDirectoryChosen`), crea e inicia el `Miner`, almacena cada canción encontrada con `saveSong` y libera los recursos en `close()`. Publica su estado (la base de datos disponible, si hay una minería en curso y la última canción encontrada) para que la vista lo observe.

El usuario interactúa con la vista, la vista notifica al controlador, el controlador coordina al modelo y actualiza su estado, y la vista se actualiza y consulta al modelo para mostrar los datos.

3.3. Decisiones de diseño

- **Desacoplamiento del Miner.** El `Miner` se comunica mediante notificaciones, de modo que puede reutilizarse sin depender del almacenamiento ni de la interfaz.
- **Minería asíncrona.** La minería se ejecuta de forma independiente de la interfaz para que esta no se bloquee. Como una nueva minería reemplaza a la anterior, el controlador debe ignorar los resultados de las mineras obsoletas.
- **Modelo de datos.** Las entidades son las canciones (rolas), los álbumes y los intérpretes. Los álbumes se identifican por el directorio que contiene a las canciones y no por la etiqueta de álbum.
- **Interfaz gráfica.** El diseño de la interfaz gráfica fue fuertemente inspirado por **Strawberry Music Player**, la preferencia personal para reproducción y administración de música.

3.4. Diagramas UML

3.4.1. Diagrama de secuencia: proceso de minería

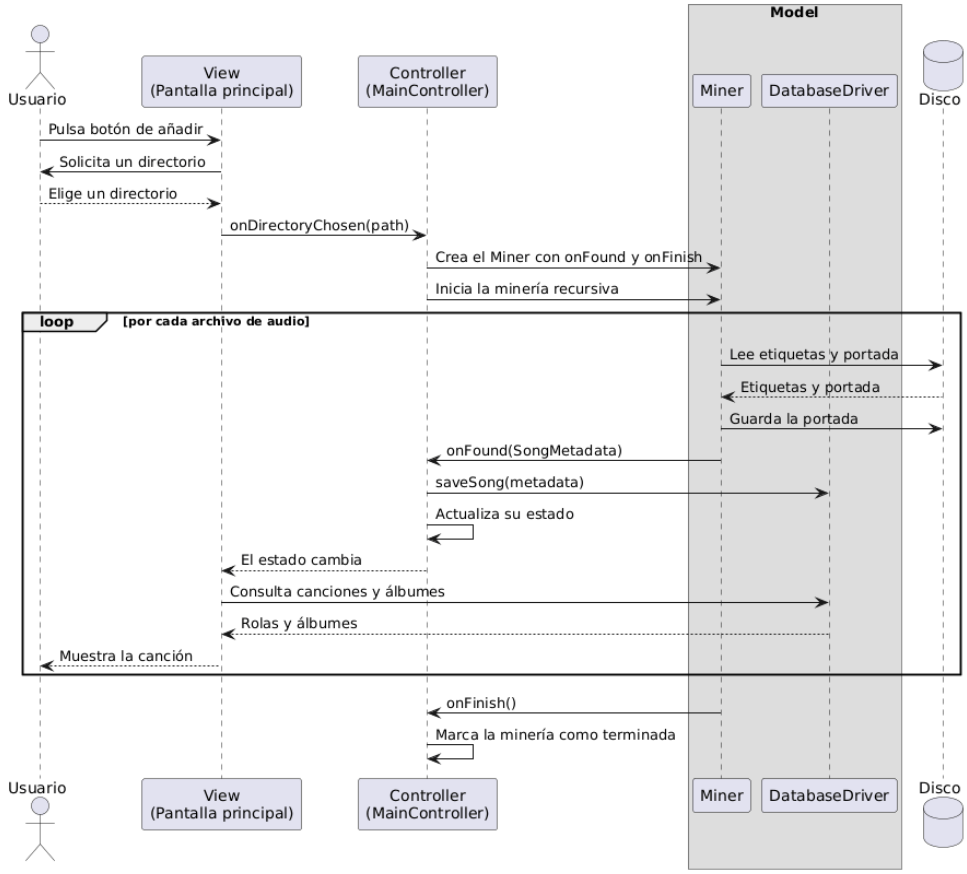


Figura 2: Diagrama de secuencia del proceso de minería de archivos.

3.4.2. Diagrama de arquitectura MVC

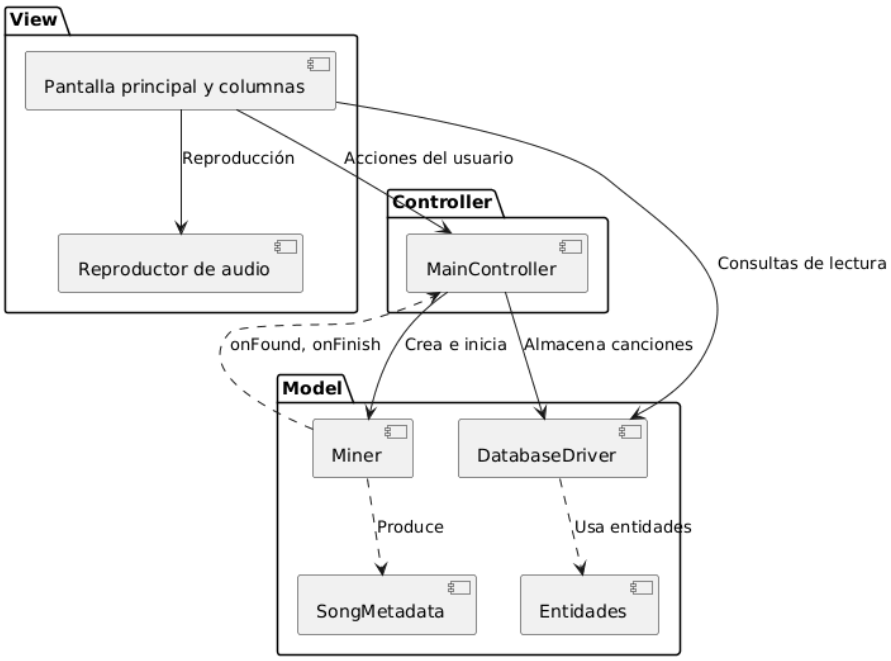


Figura 3: Diagrama de arquitectura MVC del sistema Voyager.

4. Desarrollo

El desarrollo consistió en elegir las tecnologías con las que se realizó cada componente del diseño.

4.1. Tecnologías generales

Se seleccionó **Kotlin** por su sintaxis concisa, su similitud a Java, su paradigma orientado a objetos, su manejo de funciones fuera de clases y por características como las corrutinas, que permiten manejar operaciones de entrada y salida pesadas de manera asíncrona. La interfaz se construyó con **Compose Multiplatform** y **Material 3**, que permiten crear una interfaz reactiva.

4.2. Obtención de datos: Miner

El recorrido recursivo del directorio se implementó con `File.walkTopDown()`, filtrando los archivos `.mp3` y `.flac`. Las etiquetas (título, artista, álbum, pista, año y género) se leen con la biblioteca **JAudioTagger**. Las carátulas se extraen del archivo, se convierten a JPEG con `javax.imageio.ImageIO` y se guardan en un directorio de caché.

Para evitar que la interfaz se congele durante el escaneo del disco se utilizó un **CoroutineScope** con `Dispatchers.IO`. Las notificaciones `onFound` y `onFinish` del diseño se implementaron como funciones de Kotlin recibidas por el constructor de la clase.

4.3. Almacenamiento y consulta: DatabaseDriver

Se utilizó **SQLite** mediante **JDBC**. La base de datos es un archivo (`songlibrary.db`) en el directorio `~/.local/share/voyager/`, cuyo esquema se crea la primera vez que se ejecuta el programa. Las clases de datos en `DbDataClasses.kt` incluyen métodos `fromSqlResult` para mapear directamente los registros de la base de datos a objetos de Kotlin.

4.4. Presentación: interfaz de usuario

Cada columna de la vista es una función composable de Compose Multiplatform. El estado visual (canción y álbum seleccionados, panel visible y reproductor activo) se conserva con `remember` y `mutableStateOf`. El selector de directorios utiliza `JFileChooser` de Swing, y la reproducción de audio se realiza con la biblioteca **Gadulka**.

4.5. Coordinación: MainController

El `MainController` es una clase de Kotlin independiente de los composables, que solo utiliza `mutableStateOf` (del runtime de Compose) para publicar su estado: la base de datos disponible, si hay una minería en curso y la última canción encontrada, de modo que Compose actualiza la interfaz cuando cambian. Como las notificaciones del `Miner` se ejecutan en un hilo distinto al de la interfaz, cada minería recibe un número de ejecución (`runNumber`) y los resultados de las ejecuciones obsoletas se descartan.

5. Conclusiones

La implementación del proyecto Voyager permitió consolidar conocimientos en arquitectura de software, bases de datos relacionales, programación concurrente y manejo de sistemas de archivos. El uso del modelo de cascada facilitó una comprensión clara de los requerimientos antes de iniciar el código, lo que redujo la necesidad de refactorizaciones estructurales, aunque inevitablemente se tuvieron que modificar varias partes del modelo ya definidas por no pensar en alguna limitación o por la existencia de errores clave.

El uso del patrón MVC permitió que la lógica de minería y de acceso a los datos sea independiente de la representación visual, y que el controlador concentre la coordinación entre ambas.

La mayor limitación del proyecto fue la falta de creación de pruebas unitarias. Con ellas, el desarrollo hubiera sido acelerado. Las pruebas de correcto funcionamiento se realizaron utilizando la interfaz gráfica misma y verificando la inserción adecuada dentro de la base de datos utilizando el programa **DBeaver**.

Como trabajo futuro se sugiere:

- Lograr que el método `stop()` del `Miner` interrumpa realmente el recorrido en curso; actualmente los resultados de una minería detenida solo se descartan en el controlador.

- Hacer que todas las consultas de la vista pasen por el controlador, en lugar de que las columnas accedan directamente al `DatabaseDriver`.
- Implementar pruebas unitarias para garantizar el adecuado desarrollo del programa.